

โปรแกรม sequential-search.c

```
bool Duplicate(int i, int Data1) // Check Duplication Data
{
    int j;
    for (j = 1; j <= i; j++)
    {
        if (Data1 == Data[j])
            return (true);
    }
    return (false);
}
```

ใช้ตรวจสอบว่าค่าที่ต้องการเพิ่มเข้าไปใน Array ซ้ำกับข้อมูลที่มีอยู่แล้วหรือไม่
โดยวนลูปเปรียบเทียบค่าที่รับเข้ามากับสมาชิกใน Array ตั้งแต่ตำแหน่งแรกจนถึงตำแหน่งที่กำหนด
หากพบค่าซ้ำจะคืนค่า true ถ้าไม่พบจะคืนค่า false

```
void PrepareRawKey(int N)
{
    int i, j, temp;
    srand(time(NULL)); // for difference random number in
rand()
    for (i = 1; i <= N; i++)
    {
        temp = (rand() % 89) + 10; // random difference
number 10..99
        while (Duplicate(i - 1, temp)) // Loop if Still
Duplicate
            temp = (rand() % 89) + 10; // random again
        Data[i] = temp; // Keep new Number
    } // End for
} // End Fn.
```

ใช้สร้างข้อมูลสุ่มจำนวน N ตัวและเก็บลงใน Array Data[] โดยสุ่มตัวเลขช่วง 10-98
และเรียกใช้ฟังก์ชัน Duplicate() เพื่อตรวจสอบว่าค่าที่สุ่มมาไม่ซ้ำกับค่าที่มีอยู่แล้ว
หากซ้ำจะสุ่มใหม่จนกว่าจะได้ค่าที่ไม่ซ้ำ จากนั้นจึงบันทึกลง Array

```
void DispKey(int N)
{
    int i;
    for (i = 1; i <= N; i++)
        printf("(%2d)", i); // Show Subscript i
    printf("\n");
    for (i = 1; i <= N; i++)
        printf(" %2d ", Data[i]); // Show Data[]
    printf("\n"); }
}
```

ใช้แสดงข้อมูลทั้งหมดใน Array ออกทางหน้าจอ โดยแสดงทั้งลำดับตำแหน่ง (Index) และค่าของข้อมูลในแต่ละตำแหน่ง เพื่อให้ผู้ใช้เห็นข้อมูลที่ถูกสร้างขึ้นภายในโปรแกรม

```
bool SequentialSearch(int Key1)
{
    int i;
    Times = 0; // Initial Time for search
    i = 1;
    while (i <= N)
    {
        Times++; // Count Searching Time
        if (Key1 == Data[i])
            return (true); // if found
        i++;
    } // End while
    return (false); // If not found
} // End Fn.
```

ใช้ค้นหาข้อมูลแบบลำดับ (Sequential Search หรือ Linear Search)

โดยเริ่มตรวจสอบจากสมาชิกตัวแรกของ Array ไปทีละตัวจนกว่าจะพบข้อมูลที่ตรงกับค่า Key ที่ต้องการค้นหา หากพบจะคืนค่า true ทันที แต่หากตรวจสอบครบทุกตำแหน่งแล้วยังไม่พบจะคืนค่า false นอกจากนี้ยังนับจำนวนครั้งที่มีการเปรียบเทียบข้อมูลเก็บไว้ในตัวแปร Times

```
int main()
{
    printf("SEQUENTIAL SEARCH\n");

    printf("=====\n");
    ;
    N = 16;
    PrepareRawKey(N);
    printf("Raw key :\n");
    DispKey(N); // Raw key
    printf("-----\n");
    while (key != -999)
    {
        printf("\nEnter Key for Search(-999 for EXIT) = ");
        scanf("%d", &key); // Read key from KBD
        if (key != -999)
        {
            result = SequentialSearch(key); // Call Sequential
            Search
            if (result)
```

```

        printf("Result...FOUND\n"); // if found
    else
    {
        Beep(600, 600);
        printf("Result...NOT FOUND!!\n"); // if NOT
found
    }
    printf("Searching Time : %d\n", Times);
    printf("-----Searching Finished\n");
    } // End if
} // End While
return (0);
} // End Main

```

เป็นฟังก์ชันหลักที่ควบคุมการทำงานทั้งหมดของโปรแกรม เริ่มจากกำหนดจำนวนข้อมูล
สร้างข้อมูลสุ่มด้วย PrepareRawKey() แสดงข้อมูลด้วย DispKey()
จากนั้นรับค่าที่ผู้ใช้ต้องการค้นหาและส่งไปยัง SequentialSearch() เมื่อได้ผลลัพธ์จะแสดงว่า
FOUND หรือ NOT FOUND พร้อมแสดงจำนวนครั้งที่ใช้ในการค้นหา
โดยโปรแกรมจะวนรับค่าค้นหาไปเรื่อย ๆ จนกว่าผู้ใช้จะป้อน -999 เพื่อออกจากโปรแกรม

โปรแกรม binary-search.c

```

bool Duplicate(int i, int Data1) // Check Duplication Data
{
    int j;
    for (j = 1; j <= i; j++)
    {
        if (Data1 == Data[j])
            return (true);
    }
    return (false);
}

```

ใช้ตรวจสอบว่าค่าที่ต้องการนำไปเก็บใน Array ซ้ำกับข้อมูลที่มีอยู่แล้วหรือไม่
โดยวนลูปเปรียบเทียบค่าที่รับเข้ามากับสมาชิกใน Array ตั้งแต่ตำแหน่งแรกจนถึงตำแหน่งที่กำหนด
หากพบค่าซ้ำจะคืนค่า true แต่ถ้าไม่พบจะคืนค่า false

```

void PrepareRawKey(int N)
{
    int i, j, temp;
    srand(time(NULL)); // for difference random number in
rand()
    for (i = 1; i <= N; i++)
    {

```

```

        temp = (rand() % 89) + 10;    // random difference
number 10..99
        while (Duplicate(i - 1, temp)) // Loop if Still
Duplicate
            temp = (rand() % 89) + 10; // random again
            Data[i] = temp;            // Keep new Number
        } // End for
    } // End Fn.

```

ใช้สร้างข้อมูลแบบสุ่มจำนวน N ค่าและเก็บลงใน Array Data[] โดยสุ่มตัวเลขช่วง 10-98
 จากนั้นตรวจสอบการซ้ำด้วยฟังก์ชัน Duplicate() หากค่าที่สุ่มได้ซ้ำจะสุ่มใหม่จนกว่าจะได้ค่าที่ไม่ซ้ำ
 แล้วจึงบันทึกลงใน Array

```

void DispKey(int N)
{
    int i;
    for (i = 1; i <= N; i++)
        printf("(%2d)", i); // Show Subscript i
    printf("\n");
    for (i = 1; i <= N; i++)
        printf(" %2d ", Data[i]); // Show Data[]
    printf("\n");
}

```

ใช้แสดงข้อมูลใน Array ทั้งหมดออกทางหน้าจอ โดยแสดงหมายเลขตำแหน่ง (Index)
 และค่าข้อมูลในแต่ละตำแหน่ง เพื่อให้เห็นข้อมูลที่จัดเก็บอยู่ภายใน Array อย่างชัดเจน

```

void BubbleSort(int N) // Ascending Sort
{
    int i, j, temp;
    for (i = 1; i <= N - 1; i++) // loop forward
    {
        if (Data[i] > Data[i + 1]) // if not true position
        {
            j = i + 1;                // loop backward
            while (Data[j] < Data[j - 1]) // while if remain
bubble
            {
                temp = Data[j - 1]; // swap data
                Data[j - 1] = Data[j];
                Data[j] = temp;
                j--; // count down j
            } // end while
        } // end if
    } // end for } // end Fn.

```

ใช้เรียงลำดับข้อมูลใน Array จากน้อยไปมาก (Ascending Order) ก่อนนำไปค้นหาด้วย Binary Search โดยเปรียบเทียบข้อมูลที่อยู่ติดกันและสลับตำแหน่งเมื่อพบว่าค่าด้านหน้ามากกว่าค่าด้านหลัง ทำซ้ำจนข้อมูลทั้งหมดเรียงลำดับถูกต้อง ซึ่งเป็นเงื่อนไขสำคัญที่ Binary Search ต้องมี

```
bool BinarySearch(int Key1)
{
    int L, R, Mid;
    L = 1;
    R = N;
    Times = 0; // Initial Time for search
    while (L <= R)
    {
        Mid = (L + R) / 2; // Calculate Middle
        Times++;           // Count Searching Time
        printf("L : %2d ", L);
        printf("R : %2d ", R);
        printf("Mid : %2d ", Mid);
        printf("Searching Time : %d\n", Times);
        if (Key1 == Data[Mid])
            return (true); // if found
        else
        {
            if (Key1 < Data[Mid])
                R = Mid - 1; // Move R
            else
                L = Mid + 1; // Move L
        }
    } // End while
    return (false); // If not found
} // End Fn.
```

ใช้ค้นหาข้อมูลด้วยวิธี Binary Search โดยกำหนดตำแหน่งซ้ายสุด (L) และขวาสุด (R) ของช่วงค้นหา จากนั้นคำนวณตำแหน่งกึ่งกลาง (Mid) แล้วเปรียบเทียบค่าที่ต้องการค้นหากับข้อมูลที่ตำแหน่งกึ่งกลาง หากตรงกันจะคืนค่า true แต่หากน้อยกว่าจะย้ายขอบเขตด้านขวาเข้ามา และหากมากกว่าจะย้ายขอบเขตด้านซ้ายเข้ามา ทำซ้ำจนพบข้อมูลหรือจนช่วงค้นหาหมด นอกจากนี้ยังนับจำนวนรอบการค้นหาไว้ในตัวแปร Times และแสดงค่า L, R, Mid ในแต่ละรอบเพื่อให้เห็นขั้นตอนการทำงาน

```
int main()
{
    printf("BINARY SEARCH\n");

    printf("=====\n");
    ;
```

```

N = 32;
PrepareRawKey(N);
printf("Raw key :\n");
DispKey(N); // Raw key
BubbleSort(N);
printf("-----\n");
while (key != -999)
{
    printf("Sorted Key :\n");
    DispKey(N); // Sorted Key
    printf("\nEnter Key for Search(-999 for EXIT) = ");
    scanf("%d", &key); // Read key from KBD
    if (key != -999)
    {
        result = BinarySearch(key); // Call Binary Search
        if (result)
            printf("Result...FOUND\n"); // if found
        else
        {
            Beep(600, 600);
            printf("Result...NOT FOUND!!\n"); // if NOT
found
        }
        // printf("Searching Time : %d\n",Times);
        printf("-----Searching Finished\n");
    } // End if
} // End While
return (0);
} // End Main

```

เป็นฟังก์ชันหลักที่ควบคุมการทำงานของโปรแกรม เริ่มจากกำหนดจำนวนข้อมูล สร้างข้อมูลสุ่มด้วย PrepareRawKey() แสดงข้อมูลเดิม จากนั้นเรียงลำดับข้อมูลด้วย BubbleSort() แล้วแสดงข้อมูลที่เรียงลำดับแล้ว รับค่าที่ผู้ใช้ต้องการค้นหาและส่งไปยัง BinarySearch() เมื่อได้ผลลัพธ์จะแสดงว่า FOUND หรือ NOT FOUND พร้อมแสดงรายละเอียดขั้นตอนการค้นหา โดยโปรแกรมจะทำงานซ้ำจนกว่าผู้ใช้จะป้อนค่า -999 เพื่อออกจากโปรแกรม

โปรแกรม hashing-search.c

```
typedef struct Node // Declare structure of node
{ // typedef struct Node
    int info;
    struct Node *link;
} Node; // Add Node
struct Node *Start[MaxData], *H1, *p; // Declare pointer node
Node *Allocate() // Allocate 1 node from
storage pool
{
    struct Node *temp;
    temp = (Node *)malloc(sizeof(Node)); // Allocate node by
size declare
    return (temp);
}
```

ใช้ของพื้นที่หน่วยความจำสำหรับสร้าง Node ใหม่ใน Linked List โดยใช้ malloc() จากนั้นส่งกลับ Address ของ Node ที่ถูกสร้าง เพื่อนำไปใช้เก็บข้อมูลใน Hash Table แบบ Dynamic Chaining

```
bool Duplicate(int i, int Data1) // Check Duplication Data
{
    int j;
    for (j = 1; j <= i; j++)
    {
        if (Data1 == Data[j])
            return (true);
    }
    return (false);
}
```

ใช้ตรวจสอบว่าค่าที่สุ่มได้ซ้ำกับข้อมูลที่มีอยู่ใน Array หรือไม่

โดยวนลูปเปรียบเทียบค่าที่รับเข้ามากับข้อมูลเดิมใน Array หากพบค่าซ้ำจะคืนค่า true และถ้าไม่ซ้ำจะคืนค่า false

```
void PrepareRawKey(int N)
{
    int i, temp;
    srand(time(NULL)); // for difference random number in
rand()
    for (i = 0; i < N; i++)
    {
        temp = (rand() % 989) + 10; // random difference
number 10..999
        while (Duplicate(i - 1, temp)) // Loop if Still
Duplicate
    }
}
```

```

        temp = (rand() % 989) + 10; // random again
        Data[i] = temp;           // Keep new Number
    } // End for
} // End Fn.

```

ใช้สร้างข้อมูลแบบสุ่มจำนวน N ค่าและเก็บลงใน Array Data[] โดยสุ่มตัวเลขช่วง 10-999 พร้อมตรวจสอบการซ้ำผ่านฟังก์ชัน Duplicate() หากพบค่าซ้ำจะสุ่มใหม่จนกว่าจะได้ค่าที่ไม่ซ้ำ จากนั้นจึงนำไปเก็บใน Array

```

void DispKey(int N)
{
    int i;
    for (i = 0; i < N; i++)
        printf(" %2d ", Data[i]); // Show Data[]
    printf("\n");
}

```

ใช้แสดงข้อมูลทั้งหมดที่อยู่ใน Array Data[] ออกทางหน้าจอ เพื่อให้ผู้ใช้เห็นชุดข้อมูลที่ถูกสร้างขึ้นก่อนนำไปสร้าง Hash Table

```

void CreateHead(int Head)
{
    int i;
    struct Node *p;
    for (i = 1; i <= Head; i++) // Count by Number of Head
    {
        p = Allocate();
        p->info = -1; // NULL
        p->link = NULL; // Let NEXT = NULL
        Start[i] = p; // Let Start of each node = Address of
first Node
    } // End for
}

```

ใช้สร้าง Head Node สำหรับแต่ละ Address ของ Hash Table ตามจำนวน M Address โดยกำหนดค่าเริ่มต้นของ info เป็น -1 และ link เป็น NULL จากนั้นนำ Address ของ Head Node แต่ละตัวไปเก็บใน Array Start[] เพื่อเป็นจุดเริ่มต้นของแต่ละ Bucket

```

void CreateHashTable(int N)
{
    int i;
    struct Node *H1, *p;
    for (i = 0; i < N; i++)
    {
        Addr = Data[i] % M + Lo; // Calculate Address of Key
(Addr=K mod M+10)
        H1 = Start[Addr];
    }
}

```

```

        if (H1->info == -1) // if Head Node
            H1->info = Data[i];
        else
        {
            while (H1->link != NULL)
                H1 = H1->link;
            p = Allocate(); // Add new Node
            p->info = Data[i];
            p->link = NULL;
            H1->link = p;
        } // End if
    } // End for
}

```

ใช้สร้าง Hash Table จากข้อมูลใน Array Data[] โดยคำนวณ Address ด้วยสูตร $Addr = Key \bmod M + Lo$ หาก Address นั้นยังว่างอยู่จะเก็บข้อมูลลงใน Head Node ทันที แต่หากเกิด Collision จะใช้วิธี Dynamic Chaining โดยสร้าง Node ใหม่แล้วเชื่อมต่อไว้ที่ท้าย Linked List ของ Address นั้น

```

void DispHashTable()
{
    int i;
    struct Node *H1;
    for (i = 1; i <= M; i++)
    {
        H1 = Start[i];
        printf("\nAddress %2d : ", i);
        while (H1 != NULL)
        {
            printf("%3d ", H1->info);
            H1 = H1->link; // Skip next Node
        }
    } // End for
    printf("\n");
}

```

ใช้แสดงข้อมูลทั้งหมดภายใน Hash Table โดยแสดงแต่ละ Address และข้อมูลที่เชื่อมต่อกันใน Linked List ของ Address นั้น ทำให้สามารถมองเห็นการจัดเก็บข้อมูลและการเกิด Collision ที่ถูกแก้ไขด้วย Dynamic Chaining ได้อย่างชัดเจน

```

bool SearchHash(int key)
{
    struct Node *H1;
    Addr = key % M + Lo; // Calculate Address of Key (Addr=K
mod M+lo)

```

```

H1 = Start[Addr];
Times = 0;
while (H1 != NULL)
{
    Times++; // Add Counter Times
    if (H1->info == key)
        return (true); // Found
    else
        H1 = H1->link;
}
return (false); // NOT Found
}

```

ใช้ค้นหาข้อมูลใน Hash Table โดยคำนวณ Address ของ Key ที่ต้องการค้นหาด้วยสูตร $Addr = Key \bmod M + Lo$ จากนั้นเข้าถึง Bucket ที่ตรงกับ Address นั้นและไล่ตรวจสอบข้อมูลใน Linked List ทีละ Node จนกว่าจะพบข้อมูลหรือสิ้นสุดรายการ
 พร้อมนับจำนวนครั้งที่ใช้ค้นหาเก็บไว้ในตัวแปร Times แล้วคืนค่า true เมื่อพบข้อมูล หรือ false เมื่อไม่พบข้อมูล

```

int main()
{
    printf("HASHING SEARCH(DYNAMIC CHAINING)\n");

    printf("=====\n");
    ;
    N = 32;
    M = N * 0.50; // Let M=50% of N
    PrepareRawKey(N);
    printf("Raw key :\n");
    DispKey(N); // Raw key
    printf("-----\n");
    CreateHead(M); // Create Head Node
    CreateHashTable(N);
    while (key != -999)
    {
        DispHashTable();
        printf("-----\n");
        printf("\nEnter Key for Search(-999 for EXIT) = ");
        scanf("%d", &key); // Read key from KBD
        if (key != -999)
        {

```

```

        result = SearchHash(key);
        printf("Key Address : %d\n", Addr);
        printf("Searching Time : %d\n", Times);
        printf("Result...");
        if (result)
            printf("FOUND\n"); // if found
        else
        {
            Beep(600, 600);
            printf("NOT FOUND!!\n"); // if NOT found
        }
        printf("-----\n");
    } // End if
} // End While
return (0);
} // End Main

```

เป็นฟังก์ชันหลักที่ควบคุมการทำงานทั้งหมดของโปรแกรม เริ่มจากกำหนดจำนวนข้อมูล (N) และขนาด Hash Table (M) สร้างข้อมูลสุ่มด้วย PrepareRawKey() แสดงข้อมูลเดิม สร้าง Head Node ด้วย CreateHead() และสร้าง Hash Table ด้วย CreateHashTable() จากนั้นแสดง Hash Table รับค่าที่ต้องการค้นหา และเรียก SearchHash() เพื่อตรวจสอบข้อมูล เมื่อสิ้นสุดการค้นหาจะแสดง Address ของ Key จำนวนครั้งที่ค้นหา และผลลัพธ์ว่า FOUND หรือ NOT FOUND โดยจะทำงานช้าจนกว่าผู้ใช้จะป้อน -999 เพื่อออกจากโปรแกรม